

《速通机器学习》-Attention模型和Transformer模型



北大博士后AI卢菁  
北京科技大学 工学博士

已关注

3 人赞同了该文章

1 Attention模型详解

1.1 注意力机制+的基本原理

在第13章中我们学习了时序模型LSTM，它在处理文字、语音等时序信号时有着广泛的应用。例如，可以使用两个具有不同参数的LSTM模型进行机器翻译。一个汉译英模型的架构，如图14-1所示。

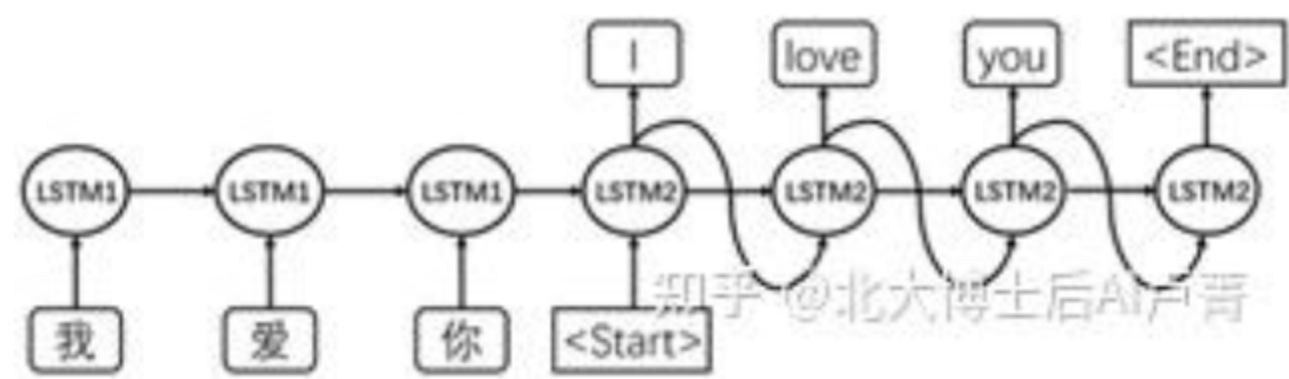


图14-1

在以上模型中，将待翻译的中文“我爱你”分成了三个词，顺序输入LSTM1，最后一个时刻的上下文信息 C 包含中文“我爱你”的完整信息，LSTM1就是编码器。

在翻译阶段，将 C 作为LSTM2的起始状态。LSTM2的第一个输入为“<Start>”，表示该位置是起始位置。输出状态 q_1 经过一个Softmax函数，预测出第一个（t=1）英文单词“I”。当 t=2 时，将上一时刻（t=1）的预测结果“I”作为该时刻的输入，预测出单词“love”。依此类推，使用LSTM2预测各个英文单词。预测类别除了正常的英文单词，还有终止字符“<End>”。预测出“<End>”字符，就表明英文句子结束，终止预测。可以看出，LSTM2就是解码器，用于把由编码器编码的中文句子解码成英文单词。

整个模型称为Encoder-Decoder模型或Seq2Seq（从序列到序列）模型。

不过，直接使用Seq2seq模型进行机器翻译会导致以下问题。

在编码阶段，输入序列的所有信息都在状态 C 中。但是，C 的描述能力有限，当输入序列较长时会丢失很多有用的信息。

由于存在长距离衰减问题，所以，当编码句子很长时，句子靠前的部分对 C 的影响将会降低。

在解码阶段，每个时刻使用前一时刻的预测结果作为输入，编码器信息 C 在 t=1 时刻作为状态信息被输入LSTM2。但是，随着序列的推移，编码信息 C 对翻译的影响越来越弱，因此，越靠后的

内容，翻译结果越差。

在翻译领域，解码阶段的词和编码阶段的词有很强的映射关系，例如“爱”和“love”。但是，Seq2seq模型无法在翻译“love”时直接使用“爱”这个词提供的信息，因为在解码阶段只能使用编码阶段的全局信息。产生上述问题的原因是，在解码阶段缺乏对编码阶段各个词的直接利用。因此，需要对模型进行改造。改造后的模型，如图14-2所示。

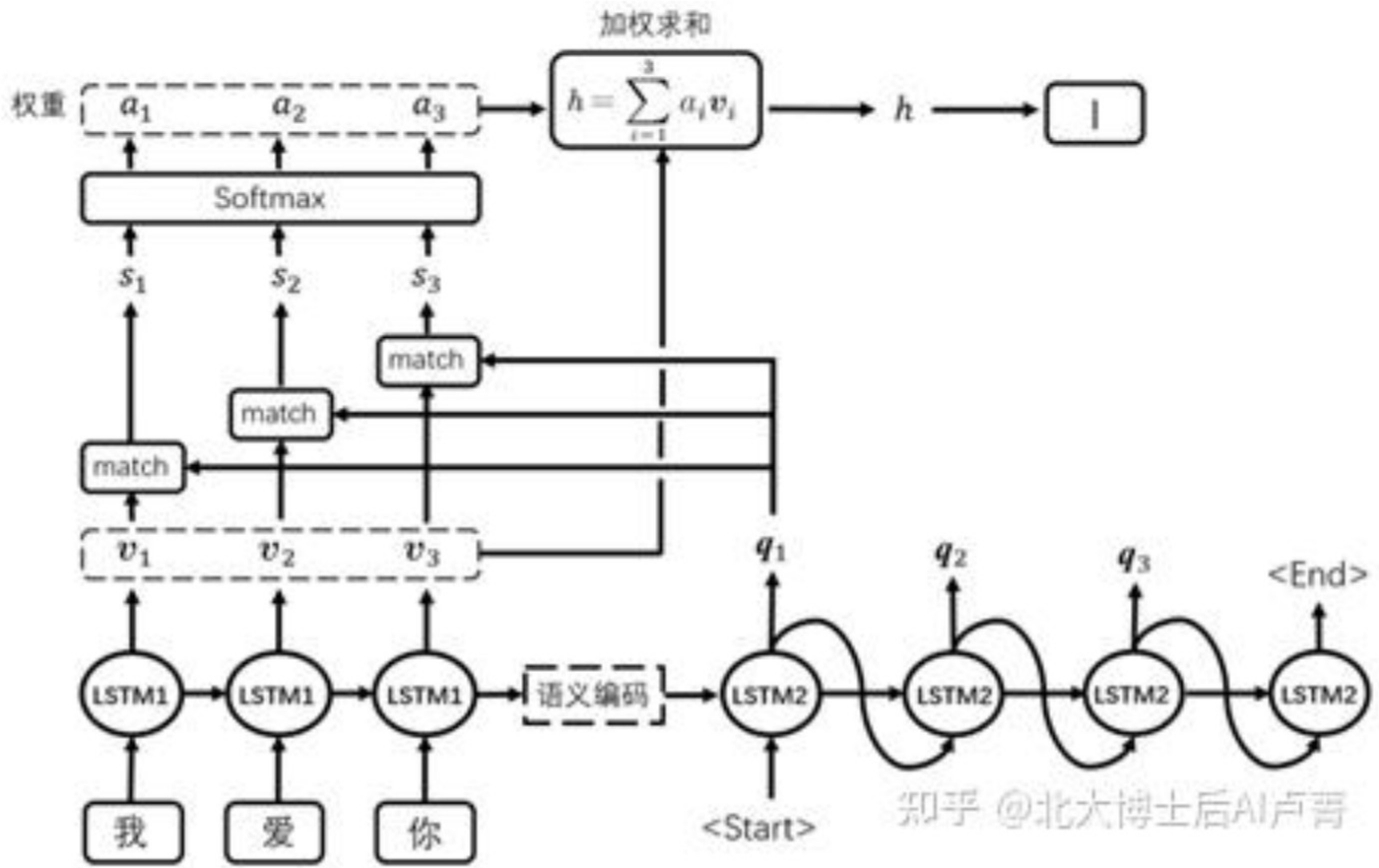


图14-2

在图14-2中：编码阶段和之前的模型没有区别，保留了各个时刻的输出向量 v ；在解码阶段，模型的预测方法有所变化，例如在 $t=1$ 时刻的预测方法如下。

首先，分别计算LSTM2在 $t=1$ 时刻的输出 q_1 ，以及 v_1, v_2, v_3 的相似度（match模块），即对 q_1 和 v_1, v_2, v_3 求内积，公式如下。

$s_1 = \langle q_1, v_1 \rangle$

$s_2 = \langle q_1, v_2 \rangle$

$s_3 = \langle q_1, v_3 \rangle$

s_1, s_2, s_3 可以理解成未被概率归一化的相似度。通过Softmax函数对 s_1, s_2, s_3 进行归一化，分别得到

$$a_1=e^{(s_1)}/(\sum_{(i=1)}^3e^{(s_i)})$$

$$a_2=e^{(s_2)}/(\sum_{(i=1)}^3e^{(s_i)})$$

$$a_3=e^{(s_3)}/(\sum_{(i=1)}^3e^{(s_i)})$$

满足 $a_1+a_2+a_3=1$ 。

a_1 、 a_2 、 a_3 为相似度得分，用于表示解码阶段 $t=1$ 时刻和编码阶段各个词之间的关系的强弱。例如，解码器在第一个时刻翻译出来的词为“ I ”，它和输入词“我”关系最近，例如 $a_1=0.95$ 、 $a_2=0.03$ 、 $a_3=0.02$ 。这一步的主要目的是让输出对输入进行聚焦，找寻最应该注意的词。

接下来，根据相似度得分对 v_1 、 v_2 、 v_3 进行加权求和，即 $h_1=a_1 v_1+a_2 v_2+a_3 v_3$ 。

最后，使用向量 h_1 通过Softmax函数来预测单词。可以看出， h_1 由最受关注的向量 v 主导，关注度就是前一时刻计算出来的 a_1 、 a_2 、 a_3 。

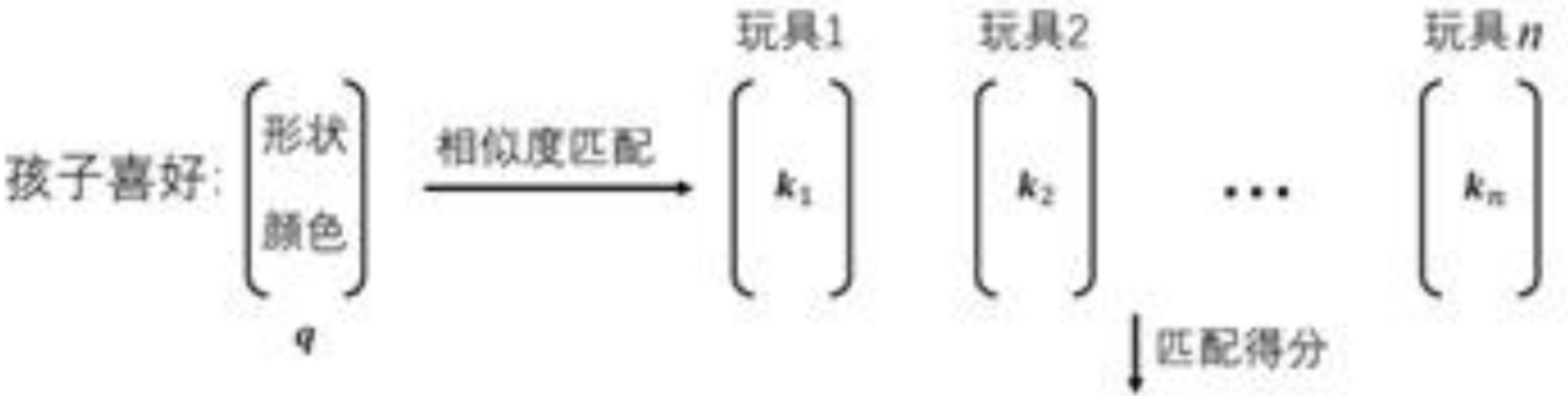
上述模型称为注意力（Attention）模型。注意力模型和人类进行翻译的方式接近。人类在进行翻译时，会先通读“我爱你”这句话，从而获得整体语义（LSTM1的输出）。在翻译阶段，人类除了会考虑整体语义，还会考虑各个输入词（“我”“爱”“你”）和当前待翻译词的映射关系（权重 a_1 、 a_2 、 a_3 ）。

1.2 Attention模型概述

14.1.1节讲解了Attention模型在Seq2Seq机器翻译中的应用。实际上，Attention作为一个近些年异常火爆的模型，有着广泛的应用。本节将介绍Attention模型的通用形式。

我们通过类比了解一下Attention模型。

家长带孩子去玩具店买玩具（一共有 n 个玩具），孩子自己挑玩具，用模型预测家长是否会在玩具店消费（只预测是否购买，不关心具体购买的玩具是什么）。每个玩具都有两类特征：1-形状,颜色,功能等；2-价格,安全性,益智性等。第一类特征主要用来吸引孩子；第二类特征则决定了家长是否会购买。设孩子自身的喜好对应于特征向量 q ，玩具的第一类特征对应于向量 k ，第二类特征对应于向量 v 。模型的结构，如图14-3所示。



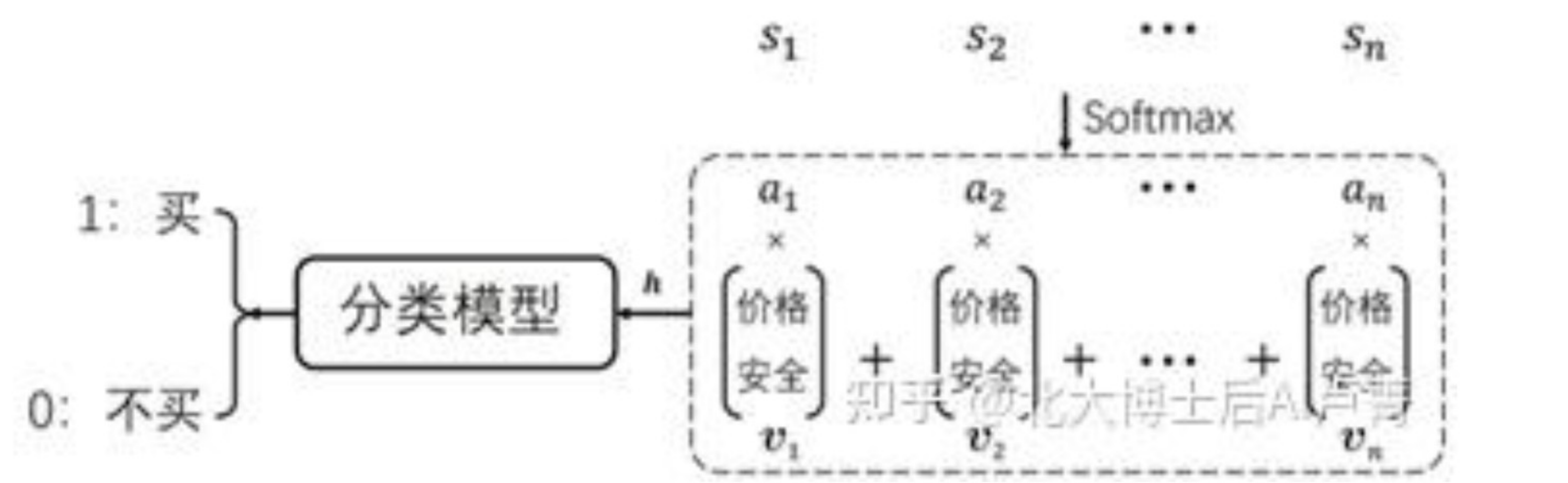


图14-3

孩子根据自身的喜好（向量 q ）挑选玩具，即计算 q 和玩具的特征 $k_1 \sim k_n$ （第一类特征）的相似度 $s_1 \sim s_n$ ，并通过Softmax函数将 $s_1 \sim s_n$ 归一化至 $a_1 \sim a_n$ 。 $a_1 \sim a_n$ 表示孩子对每个玩具的喜好的权重。接下来，通过 $a_1 \sim a_n$ 对玩具的第二类特征 $v_1 \sim v_n$ （家长角度）进行加权求和，得到向量 h ，即 $h = \sum_{i=1}^n [a_i v_i]$ 。 h 代表家长对玩具的喜好的加权和，也就是说，家长是

否购买玩具是由向量 h 决定的。

上述过程就是Attention模型的标准操作过程。Attention模型的三要素是 q 、 K 、 V 。其中， K 和 V 分别对应于向量序列 $k_1 \sim k_n$ 和 $v_1 \sim v_n$ ，即 $K = [k_1 @ \dots @ k_n]$ 和 $V = [v_1 @ \dots @ v_n]$ ，它们不仅序列长度一致，而且元素都是对应的，即 k_j 和 v_j 对应。 k_j 和 v_j 均为行向量，分别表示两类特征，因此向量长度可以不一致。当 q 为行向量时，输出就是向量 h （本章涉及的各类特征都采用行向量的形式，主要是为了书写方便）。

Attention模型的操作步骤，如图14-4所示。其中， $f(q,k)$ 用于计算相似得分（除了前面提到的内积运算，在14.1.3节还将介绍其他方法）。为了运算方便，把Attention操作记为 $h = \text{Attention}(q,K,V)$ 。

q 也可以是一个向量序列，即 $Q = [q_1 @ \dots @ q_m]$ ，此时的输出也是一个向量序列，即 $H = [h_1 @ \dots @ h_m]$ 。因此，Attention模型的通用标准公式可以写成

$$H = \text{Attention}(Q,K,V) = [\text{Attention}(q_1,K,V) @ \dots @ \text{Attention}(q_m,K,V)]$$

在这里， H 、 Q 、 K 、 V 均为向量序列，其中： H 和 Q 的序列长度一致，且各行一一对应； K 和 V 的序列长度一致，且各行一一对应。

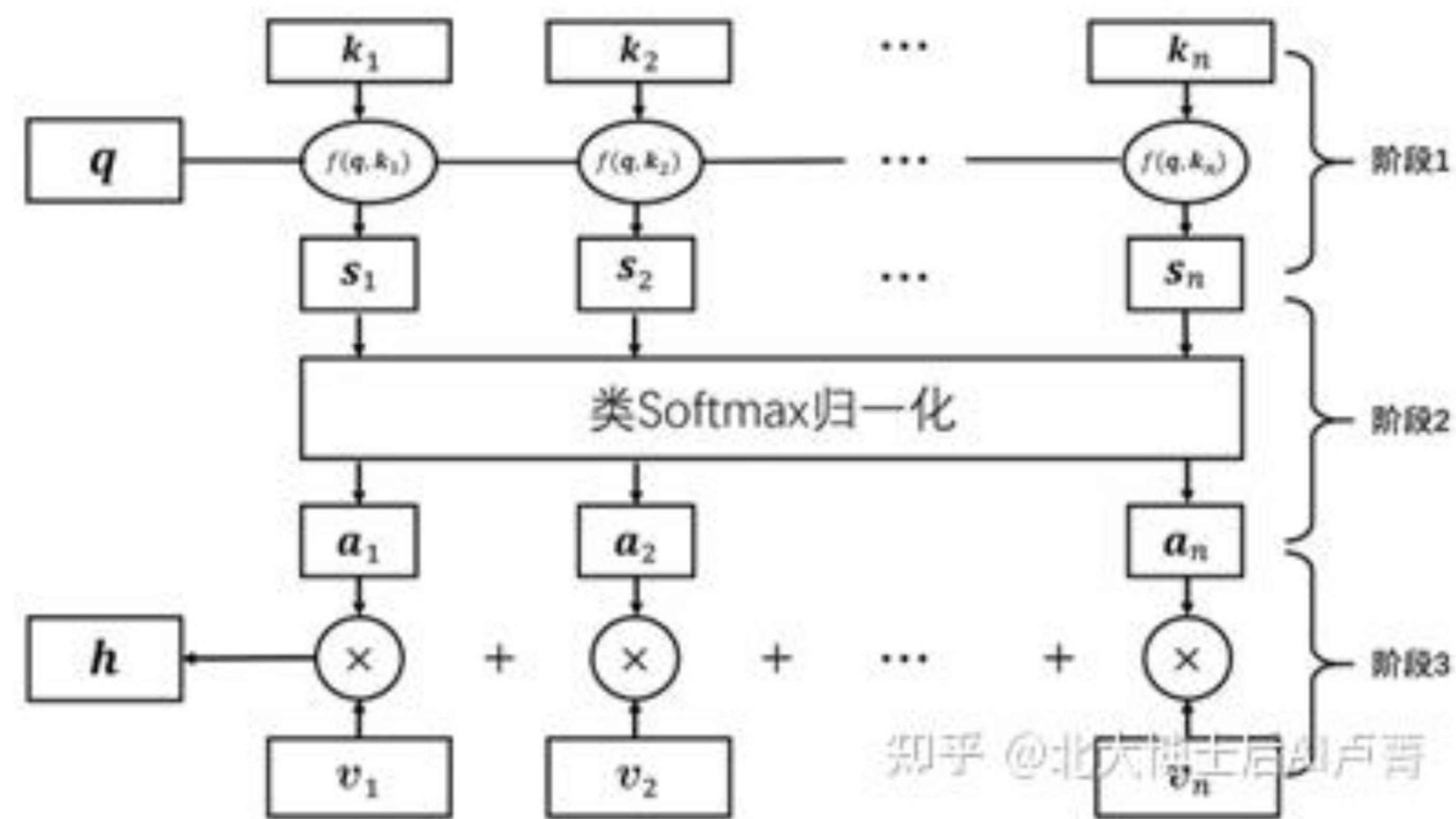


图14-4

因为Attention模型的所有向量都是行向量，所以，可以方便地将 H 写成矩阵相乘的形式，公式如下。

$H=Softmax(QK^T)V$

其中，Softmax函数分别对矩阵的每一行进行归一化操作。

在14.1.1节介绍的机器翻译模型中使用的Attention模型，其实就是

$K=V=[\blacksquare(v_1@v_2@v_3)]$

$Q=[\blacksquare(q_1@q_2@q_3)]$

1.3 Attention模型的改进形式

Attention模型求相似度，除了通过 $\langle q,k \rangle$ 直接求内积，还有很多方法，举例如下。

第一种方法，公式如下。

$s=A^T \tanh(qW+kU)$

其中，A、W 和 U 都是待学习参数。这种方法不仅避开了求内积时 q 和 k 的向量长度必须一致的限制，还可以进行不同空间的向量匹配。例如，在进行图文匹配时，q 和 k 分别来自文字空间和图像空间，可以先分别通过 W 和 U 将它们转换至同一空间，再求相似度。

第二种方法，公式如下。

$s=qWk^T$

在这里，W 为待学习参数，q 和 k 的维度可以不同。

下面介绍第三种方法。在通过 $\langle q,k \rangle$ 求相似度时，如果特征维度过高，就会导致计算出来的相似度 s 的值过大，而 s 的值过大会进一步导致归一化函数Softmax饱和，使计算出来的 a 要么接近 1，要么接近0，即加权求和退化成“胜者全拿”。因此，可以使用以下公式进行优化。

$s= \langle q,k \rangle /\sqrt{d}$

为了解决 q 和 v 的向量长度必须一致的问题，也可以使用以下公式。

$s=(qWk^T)/\sqrt{d}$

其中，d 是 q 或 v 的维度，用于避免 s 受到过高的维度的影响。

最后介绍第四种方法。在第一种方法和第二种方法中，s 也会受到过高的维度的影响，因此可以进行优化，得到以下公式。

$s= \langle qW,kU \rangle /\sqrt{d}$

其中，d 是 q 或 v 的维度。

1.4 Self-Attention模型

$H=Attention(Q,K,V)$ 有一种特殊情况，即 $Q=K=V$ ，这就是自注意力（Self-Attention）模型。

Self-Attention模型的输入只有一个序列，序列长度为 m，序列中的每个元素均为向量，对应的输出 H 是长度为 m 的向量序列。Self-Attention模型的输入和输出均为等长序列，因此，从效果的角度看，Self-Attention⁺可以取代LSTM。

当 m=4 时，Self-Attention模型如图14-5所示。

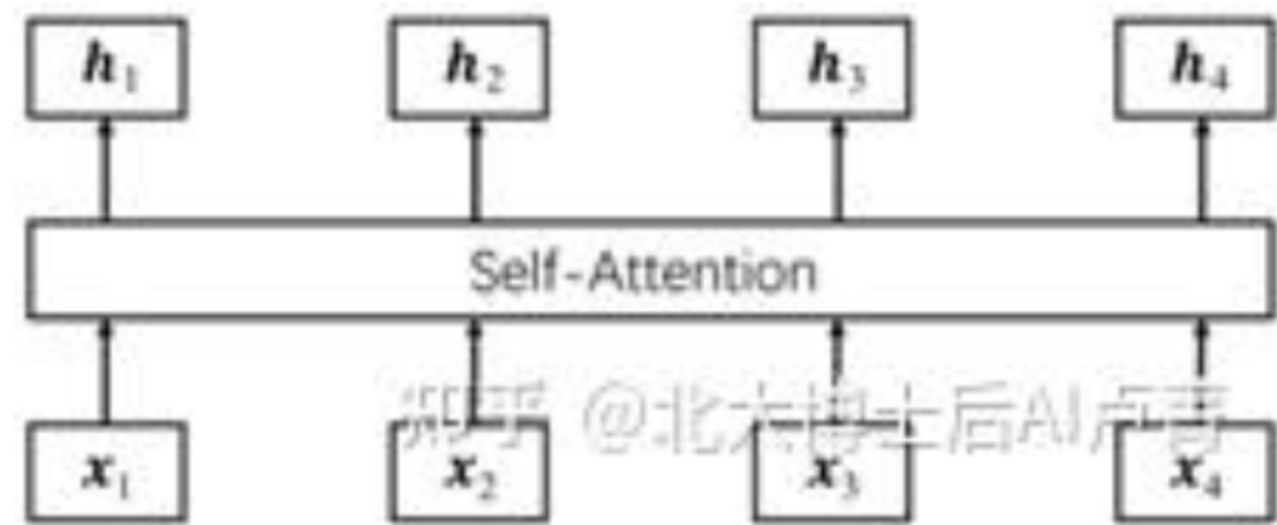


图14-5

其中， $h_i = \text{Attention}(x_i, X, X)$ ， $X = [x_1 @ x_2 @ x_3 @ x_4]$ 。可以看出，各个时刻的输出都直接与所

有时刻的输入关联起来了。在LSTM中， $t=4$ 时刻需要使用 $t=1$ 时刻的信息，因此应通过 $t=2$ 、 $t=3$ 时刻所对应的隐藏单元传递信息，距离越远，信息传递损失越大（长距离依赖问题）；而在Self-Attention中，在 $t=4$ 时刻， h_4 可以和其他所有时刻的信息直接关联。因此，Self-Attention可以解决LSTM的长距离依赖问题。

另外，LSTM在计算 t 时刻的输出时，必须把 $t-1$ 时刻的状态计算完成，每个时刻的输出和状态都依赖前一时刻，因此，LSTM无法在各个时刻并行计算（这会导致在处理大规模数据时速度受限）。而Self-Attention在所有时刻完全独立，可以并行计算 $h_1 \sim h_n$ （这是Self-Attention的一个重要优势）。

不过，Self-Attention是一个词袋模型。例如，对序列 x_1, x_2, x_3, x_4 ，有

$$h_1 = \sum_{i=1}^4 a_{1i} x_i$$

a_{1i} 为 x_1 和 x_i 归一化后的相似度。

如果打乱后面三个时刻的顺序，例如 x_1, x_4, x_2, x_3 ，则有

$$h_1 = a_{11} x_1 + a_{14} x_4 + a_{12} x_2 + a_{13} x_3 = \sum_{i=1}^4 a_{1i} x_i$$

因此，Self-Attention模型对词序不敏感。

因为语义与词序有一定的关联，所以在使用Self-Attention模型处理文本序列时会有精度方面的损失。为了解决这个问题，可先让输入序列 x_1, x_2, x_3, x_4 通过一个LSTM模型，再对输出序列使用Self-Attention模型，如图14-6所示。

另一种方法是，使每个位置对应于一个位置向量，将输入和位置向量按位相加后输入模型，如图14-7所示。其中，位置向量 $p_1 \sim p_4$ 是通过训练得到的。

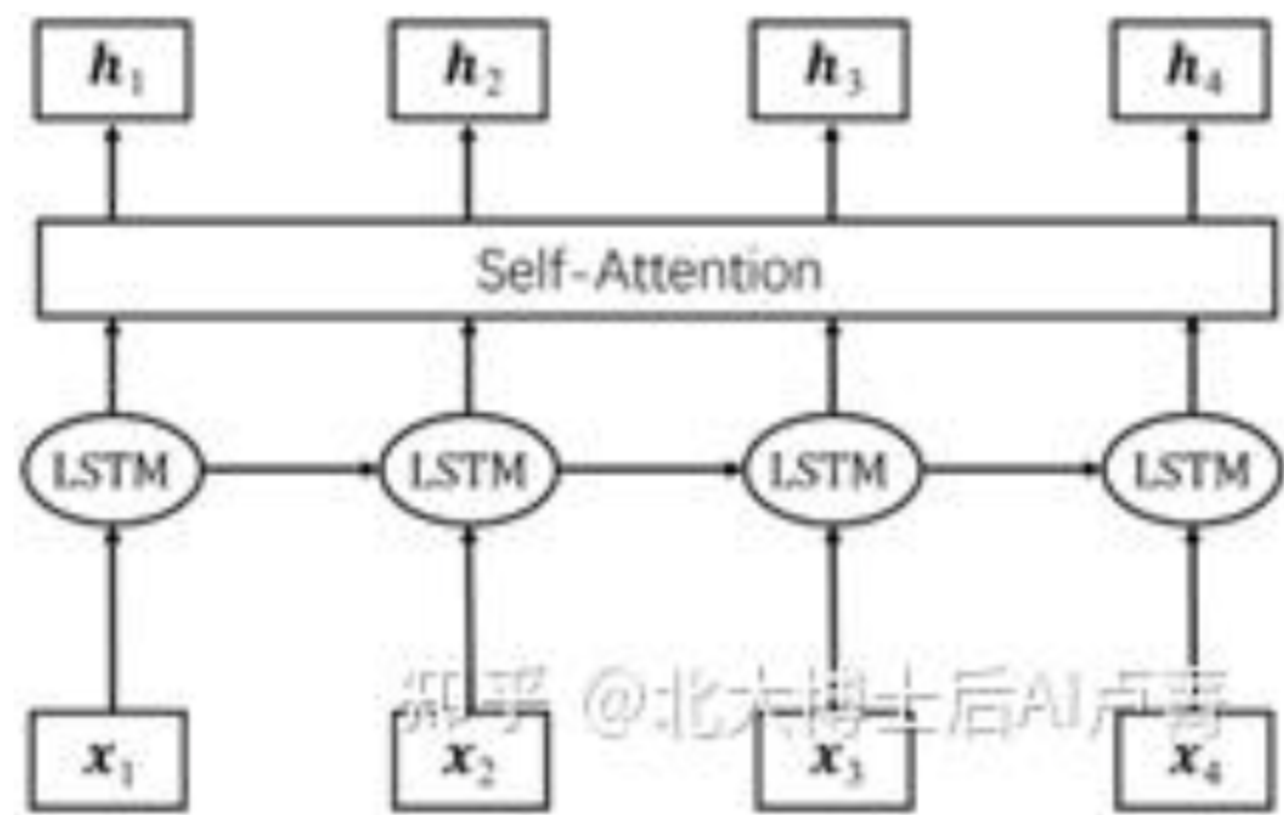


图14-6

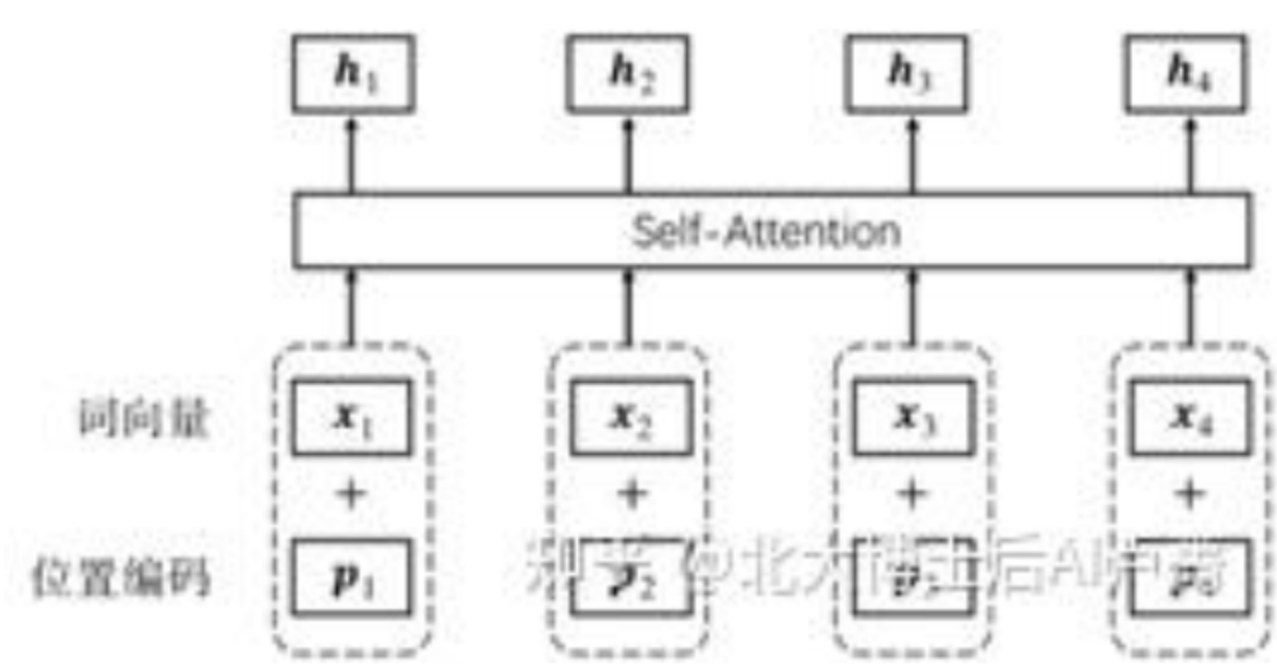


图14-7

1.5 Multi-Head Attention模型

Attention还有一个重要的变种模型，就是多抽头（Multi-Head）Attention模型。

在标准的Attention模型 $\text{Attention}(Q,K,V)$ 中对 Q 和 K 进行相似度匹配，相当于在特定的角度上计算相似度， Q 和 K 所在的向量空间即为观察角度，并且有且仅有一个观察角度。然而，实际情况要复杂得多。例如，“小明养了一只猫，它特别调皮可爱，他非常喜欢它”这句话是Self-Attention的输入序列，其中每个词都和其他词有一个对应的相似度得分。“猫”从指代的角度看，与“它”的匹配度最高，但从属性的角度看，与“调皮”“可爱”的匹配度最高。由于标准的Attention模型无法处理这种多语义的情况，所以，需要将向量序列 $Q、K、V$ 多次转换至不同的语义空间。下面讨论一下如何对标准的Attention模型进行多语义匹配改进。Self-Attention是Attention模型的一个特例，因此改进方法在Self-Attention中也适用。

通过特征变换 $W_1^Q、W_1^K、W_1^V$ 将 $Q、K、V$ 转换至特定语义空间1，公式如下。

$$Q_1=QW_1^Q=[\begin{matrix} q_1 & W_1^Q @ : & @q_m & W_1^Q \end{matrix}]$$

$$K_1=KW_1^K=[\begin{matrix} k_1 & W_1^K @ : & @k_n & W_1^K \end{matrix}]$$

$$V_1=VW_1^V=[\begin{matrix} v_1 & W_1^V @ : & @v_n & W_1^V \end{matrix}]$$

$Q、K、V$ 都是向量序列，因此特征变换就是在各时序上的向量变换。 $W_1^Q、W_1^K、W_1^V$ 均为矩阵，矩阵中的元素均为待学习参数。

在转换后的空间中进行Attention运算，即 $[\text{head}]_1=\text{Attention}(Q_1,K_1,V_1)$ 。在这里，计

算相似度时采用 $s=(\langle q,k\rangle)/\sqrt{d}$ 。以上三式也可以写成如下形式。

$$[\text{head}]_1=\text{Softmax}((Q_1 K_1^T)/\sqrt{d}) V_1$$

需要注意的是， $[\text{head}]_1$ 也是向量序列，向量长度和 Q 一致，即

$$[\text{head}]_1=[\begin{matrix} h_{1,1} & @ : & @h_{(1,m)} \end{matrix}]$$

$h_{1,1} \sim h_{1,m}$ 均为行向量，向量长度和 $v_1 W_1^V \sim v_n W_1^V$ 一致。

使用特征变换 W_2^Q 、 W_2^K 、 W_2^V 将 Q 、 K 、 V 转换至特定语义空间²，公式如下。

$$Q_2 = QW_2^Q$$

$$K_2 = KW_2^K$$

$$V_2 = VW_2^V$$

接下来，依然对转换后的空间进行Attention运算，即 $[\text{head}]_2 = \text{Attention}(Q_2, K_2, V_2) =$

$\text{Softmax}((Q_2 K_2^T) / \sqrt{d}) V_2$ 。其中， $[\text{head}]_1$ 和 $[\text{head}]_2$ 为从不同角度进行Self-Attention运算的结果。

同理，可以算出 $[\text{head}]_3, \dots, [\text{head}]_c$ 。 $[\text{head}]_1, \dots, [\text{head}]_c$ 为多个不同语义空间的注意力计算结果，我们需要把它们串联起来。 $[\text{head}]_1, \dots, [\text{head}]_c$ 均为向量序列，因此这里进行的是同一时间位的向量串联，串联结果仍然是向量序列，如下式所示。

$$\text{concat}([\text{head}]_1, [\text{head}]_2, \dots, [\text{head}]_c) = [\begin{matrix} \text{concat}(h)_{1,1}, \dots, h_{c,1} \\ \text{concat}(h)_{1,m}, \dots, h_{c,m} \end{matrix}]$$

concat 表示向量串联操作。例如，有 $a=[1,2]$ ， $b=[3,4]$ ，那么 $\text{concat}(a,b)=[1,2,3,4]$ 。

因为各抽头串联在一起后的维度可能比较高，所以，通过 W^O 对串联结果进行一次线性变换，可以实现降维和使各抽头的信息融合的目的，得到最终结果。

这就是多抽头Attention模型，其公式如下。

$$\begin{aligned} \text{MultiHead}(Q,K,V) &= \text{concat}([\text{head}]_1, [\text{head}]_2, \dots, [\text{head}]_n) W^O \\ &= [\begin{matrix} \text{concat}(h)_{1,1}, \dots, h_{n,1} \\ \text{concat}(h)_{1,m}, \dots, h_{n,m} \end{matrix}] W^O \end{aligned}$$

多抽头Attention模型的整个运算流程，如图14-8所示。

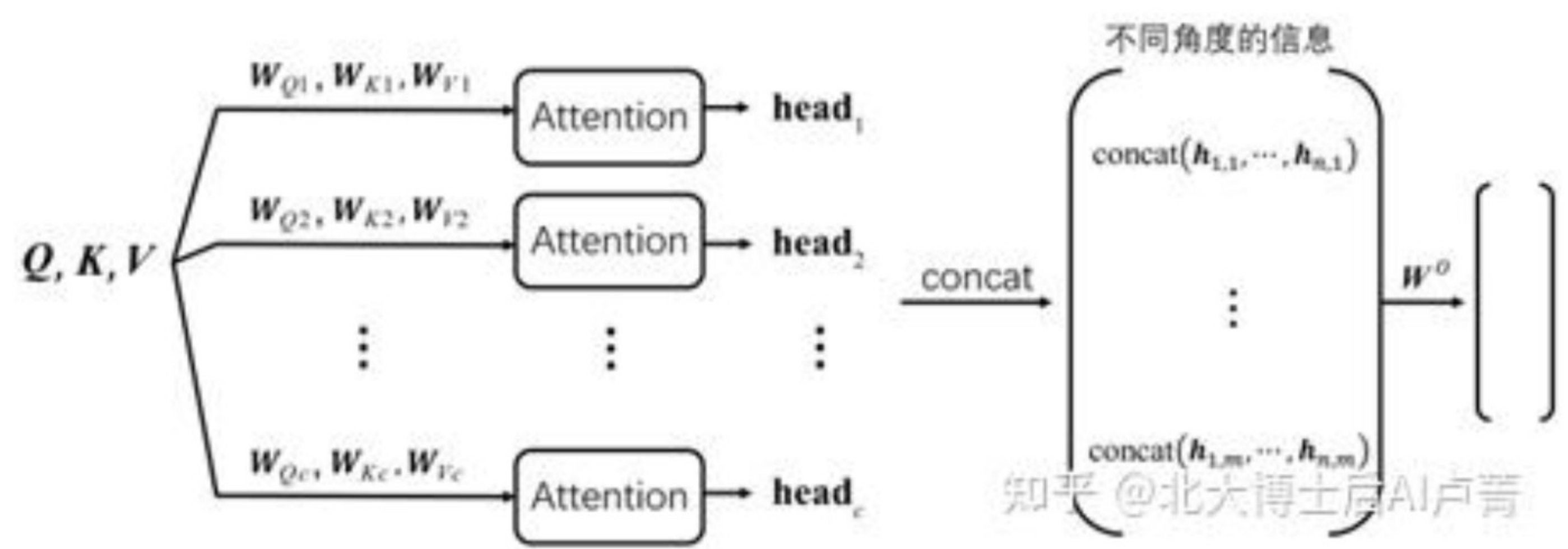


图14-8

与经典Attention模型相比，多抽头Attention模型从不同的角度进行注意力计算，在语义较为复杂的场景中，仍然可以从多个角度提取特征。

在使用多抽头Attention模型时，如果 $Q=K=V$ ，那么模型也称为多抽头自注意力（Multi-Head Self-Attention）模型。仅从输入和输出的角度看，多抽头自注意力模型的输入是一个序列向量，输出是一个长度与输入长度相同的序列向量（与LSTM相同）。

2 Transformer模型原理

Transformer是由Google Brain团队于2017年在论文Attention is all you need中提出并用于机器翻译的强大模型。Transformer采用Seq2seq模型的编码—解码框架，为了提高模型能力，编码和解码模块均采用多层Encoder和Decoder。具体的，编码器和解码器都由6个小块组成，最后一级编码块（Encoder Block）的输出是各级解码块（Decoder Block）的输入，如图14-9所示。每个编码块和解码块不再由RNN组成，而由注意力模型和前馈网络组成，如图14-10所示。

重点分析一下Encoder模块，其架构如图14-11所示。除了词向量，还在第一层增加了位置向量，其作用是避免Self-Attention的词袋模型问题，相当于给模型注入了顺序信息。

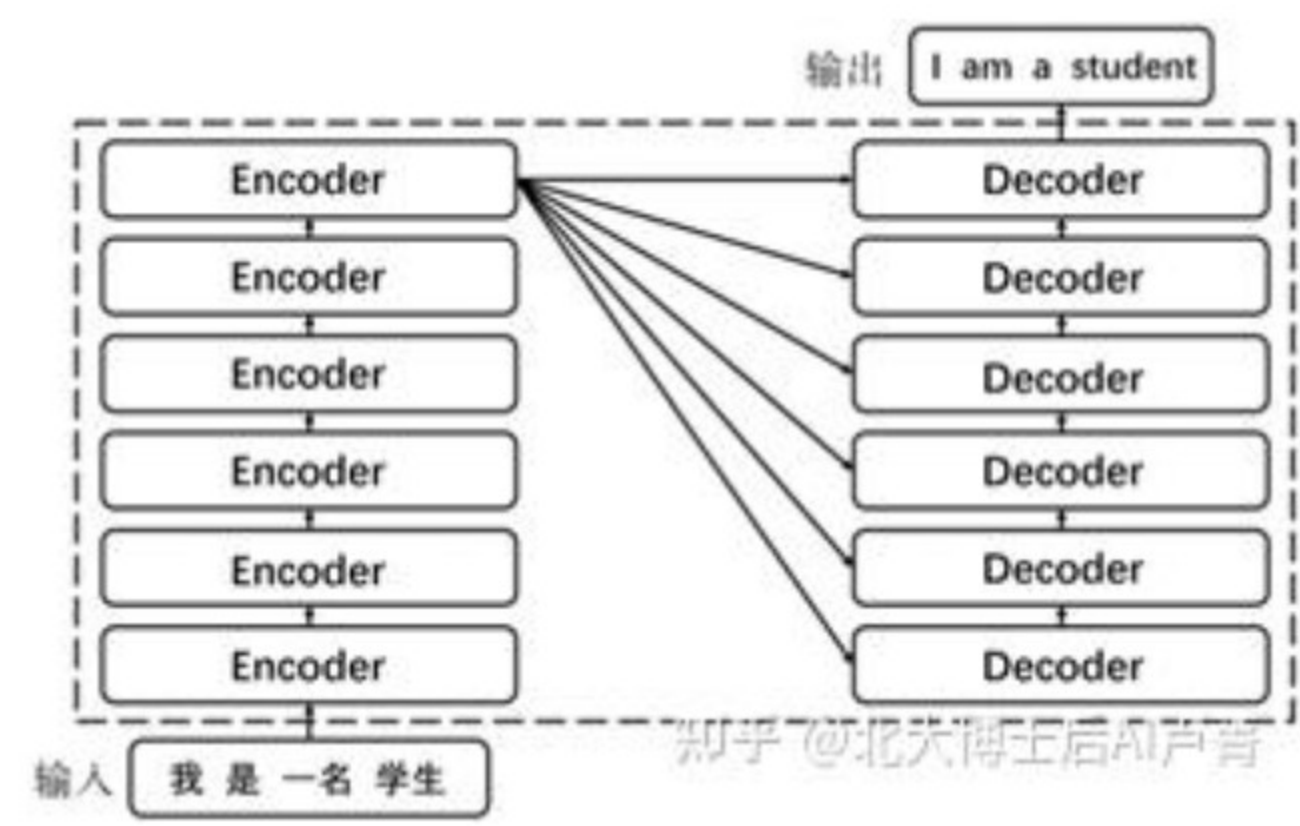


图14-9

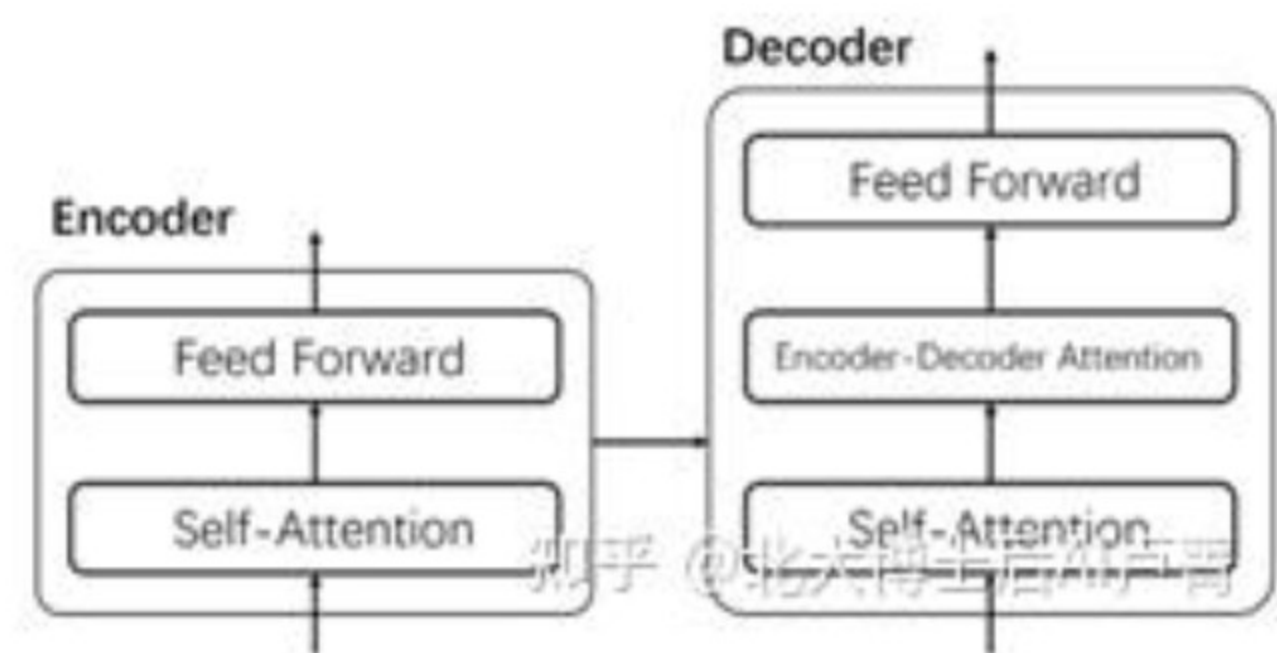


图14-10

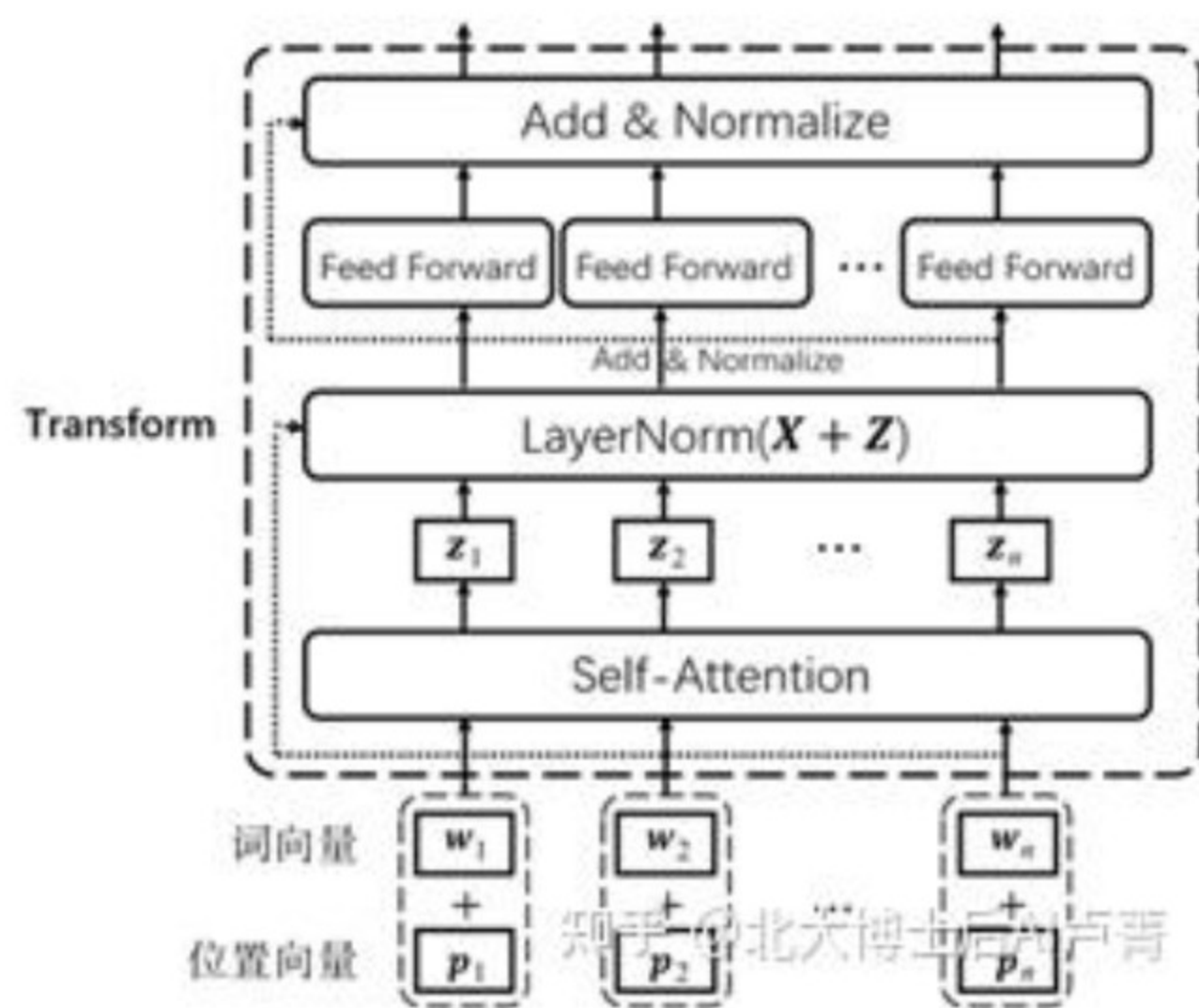


图14-11

Self-Attention模型的作用是提取语义级别的信息（不存在长距离依赖问题）。前馈网络（Feed Forward）在各个时序上对特征进行非线性变换，从而提高网络的表达能力。因为Self-Attention模型本身不会改变向量空间，只会对输入重新进行加权组合，所以，在使用将多个Encoder串联在一起的方法构造模型时，如果不使用前馈网络进行向量空间的变换，那么多个Encoder的效果和单个Encoder的效果是等价的（达不到预期效果）。

Encoder模块采用Shortcut技术解决了层数过多导致的**梯度消失问题**⁺。LayerNorm用于提高网络的训练速度和防止过拟合。

Transformer是深度学习的集大成之作，融合了多项实用技术，不仅在自然语言处理领域的许多问题中得到了应用，在计算机视觉、推荐系统等领域也得到了广泛应用。14.3节将重点介绍Transformer在自然语言处理领域的一个重要应用——BERT模型。

3 BERT模型及应用

“BERT”的全称是“Bidirectional Encoder Representation from Transformers”，即“双向Transformer解码器”。BERT是自然语言处理领域的一种龙骨模型，用于提取各类任务的基础特征，其结构如图14-12所示。

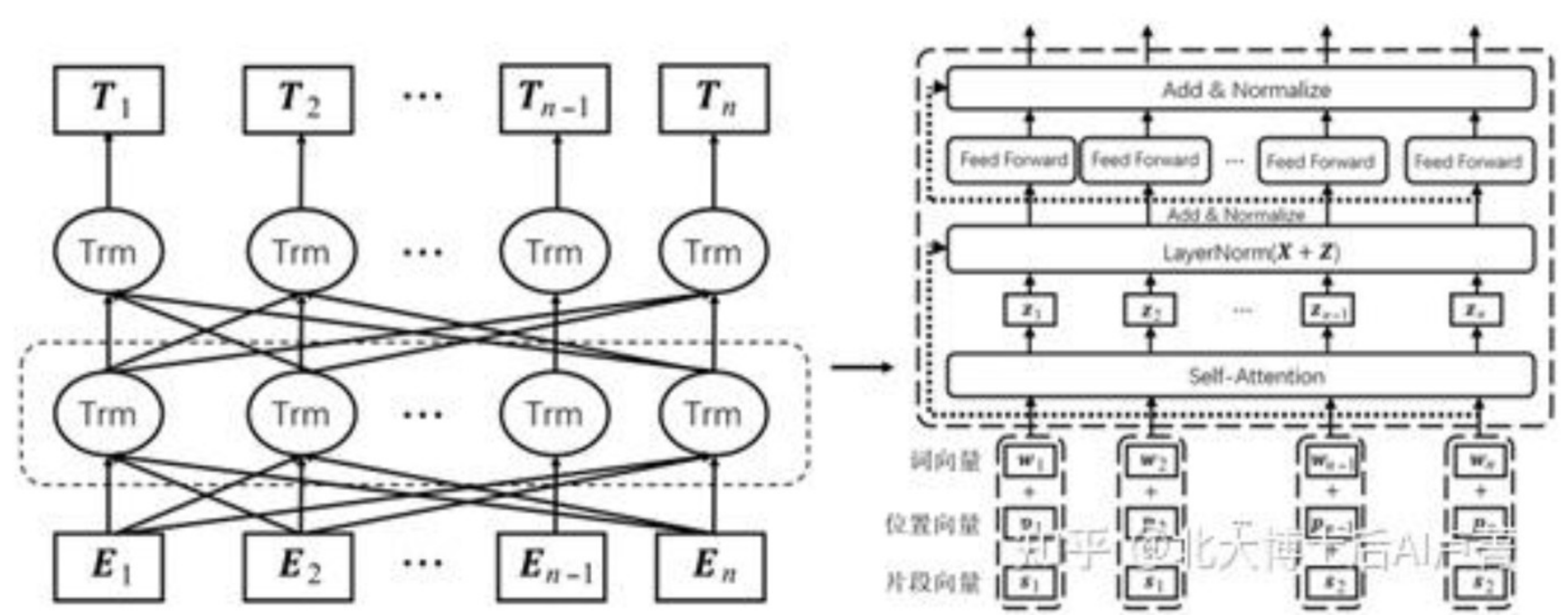


图14-12

“Trm”就是14.2节讨论的Transformer的Encoder模块。 $E_1 \sim E_n$ 为输入句子中的词或字（中文为汉字序列，英文为单词序列，并在原句的基础上添加了一些标志位，后面将具体讲解），由词（字）向量 $w_1 \sim w_n$ 、位置向量 $p_1 \sim p_n$ 及片段向量 s_1 和 s_2 相加组成。由于BERT使用的Transformer模型的核心Self-Attention是一个词袋模型，会丢失词序信息，所以，需要让每个字的位置对应于一个embedding（Position embedding，即 p_i ）。因为模型在训练时会以两个句子为输入进行分类，所以，片段向量 s_1 和 s_2 对应于不同的句子。最终的输入向量为 $E_i = w_i + p_i + s_1$ 或 $E_i = w_i + p_i + s_2$ 。经过多个Trm层，每个位置均会输出相应的向量 $T_1 \sim T_n$ ，这些向量可以作为对应位置的词（字）向量。特别地，模型的输入 E_1 为特殊字符 “[CLS]”，它所对应的输出向量 T_1 是整个句子的embedding，例如在进行文本分类时将 T_1 作为后续分类器的输入。

与Word2vec等相比，BERT在产生词（字）所对应的向量时，不仅会考虑词（字）本身，还会考虑句子的上下文。因此，词（字）所对应的向量不再是一成不变的，而是BERT模型根据其上下文动态计算出来的。例如，手机“苹果”和水果“苹果”的上下文差异较大，对应的 T 的差异也比较大。

训练BERT模型使用的语料类似于Word2vec，无须特别进行标注，只需对文本进行常规的预处理。BERT模型有两种学习方法，分别是单词级别的和句子级别的。

在单词级别的学习方法中，BERT模型使用“mask”字符以15% 的概率随机遮挡输入字符，然后通过上下文预测“mask”位置为何词。需要注意的是，对于输入，除了句子本身，还要在句首加上 “[CLS]”来表示句子的整体特征、在句尾加上 “[SEP]”作为句子的结束符。

例如，输入的句子为“我的狗叫小明”，被“mask”字符遮挡的字符为“小”，模型架构如图14-13所示。

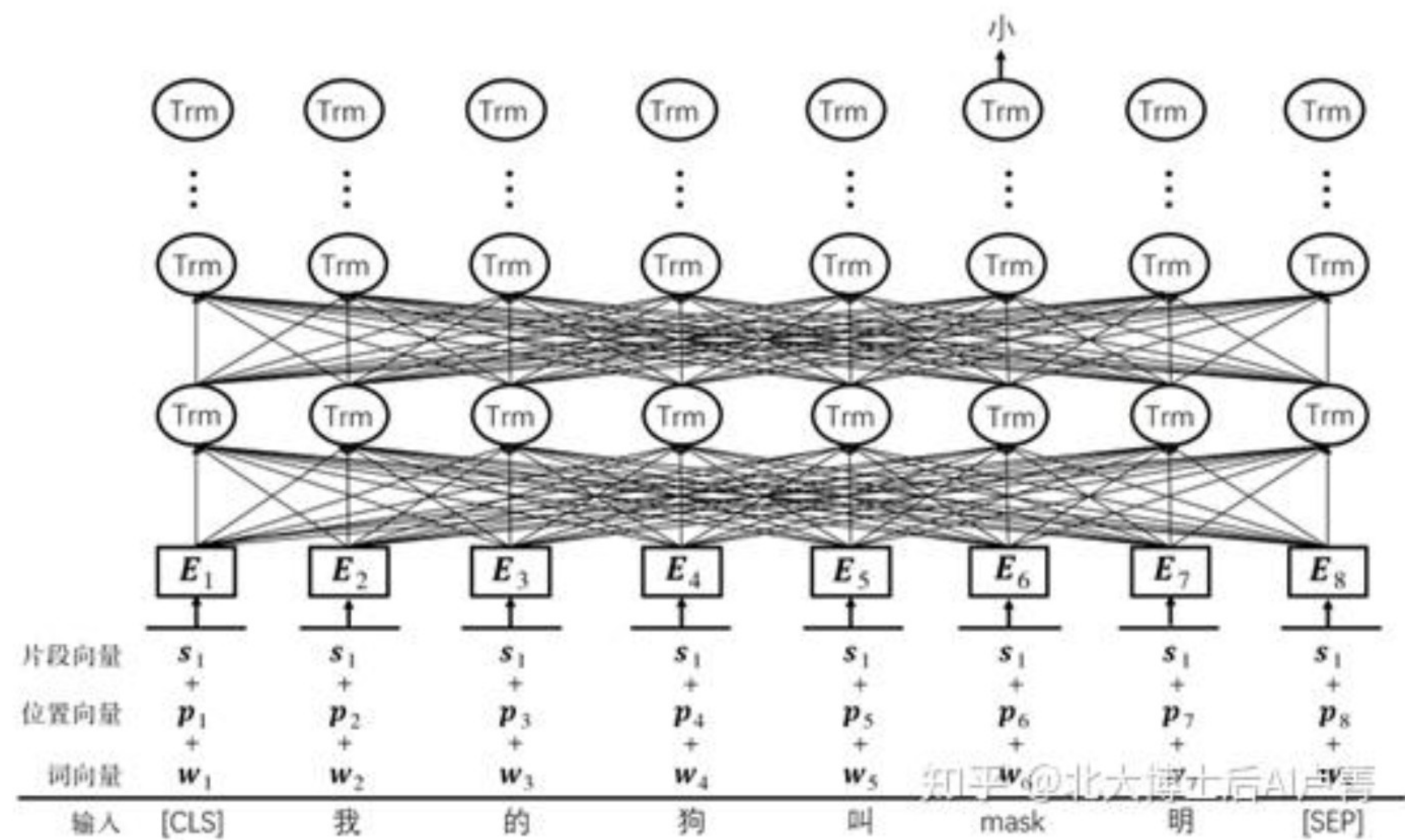


图14-13

用于遮盖的特殊标记“mask”，其作用仅仅是训练模型，在下游的自然语言处理任务中并不存在。因此，为了和后续任务保持一致，应按比例在需要预测的词的位置输入原词、随机词或“mask”。例如，对“我的狗叫小明”中的“小”字，有如下替换方式。

有80% 的概率用“mask”标记进行替换——“我的狗叫mask明”。

有10% 的概率用一个随机采样的词进行替换——“我的狗叫大明”。

有10% 的概率不进行替换——“我的狗叫小明”。

因为BERT涉及问答系统之类的任务，所以增加了第二个预训练任务，其目的是让模型理解两个句子的关联关系。在训练时，输入两个句子，第二个句子有50% 的概率是第一个句子的下一句。输入完成后，模型将预测第二个句子是否为第一个句子的下一句，如图14-14所示。第一个句子中的每个字所对应的片段向量均为 s_1 ，第二个句子中的每个字所对应的片段向量均为 s_2 。

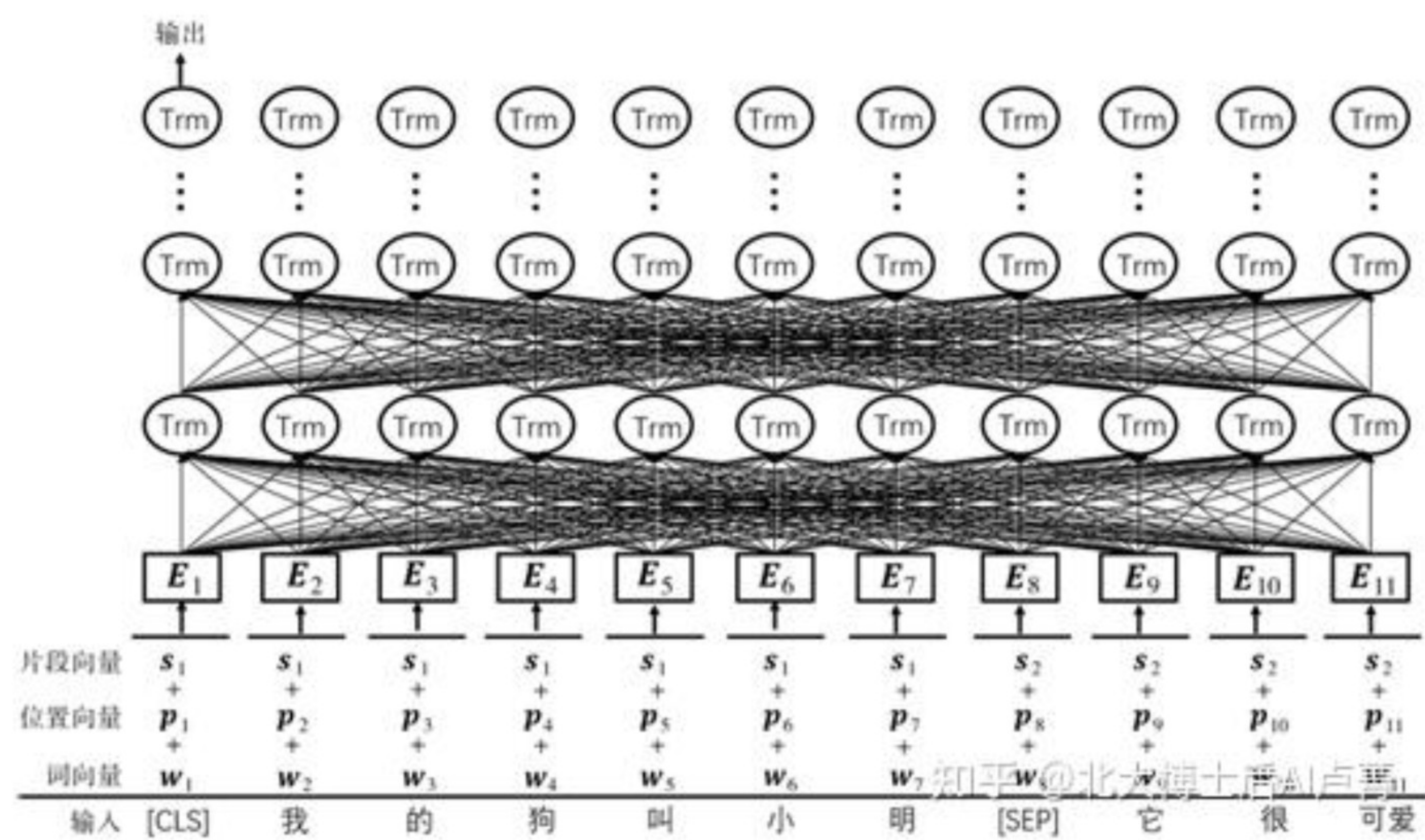


图14-14

结合上述两个目标，整个训练模型如图14-15所示。

由于训练时使用的语料和计算量都非常大，Google提供了训练好的网络，我们可以直接下载。Google提供了Base和Large两个版本的BERT模型，对应的参数如下。

Base版本：L=12，H=768，A=12，参数总量 1.1×10^8 个。

Large版本：L=24，H=1024，A=16，参数总量 3.4×10^8 个。

H 为输入（输出）词（字）所对应的embedding的大小，即向量 E 和 T 的长度。A 为Multi-Head 的个数。L 为Trm层的个数。

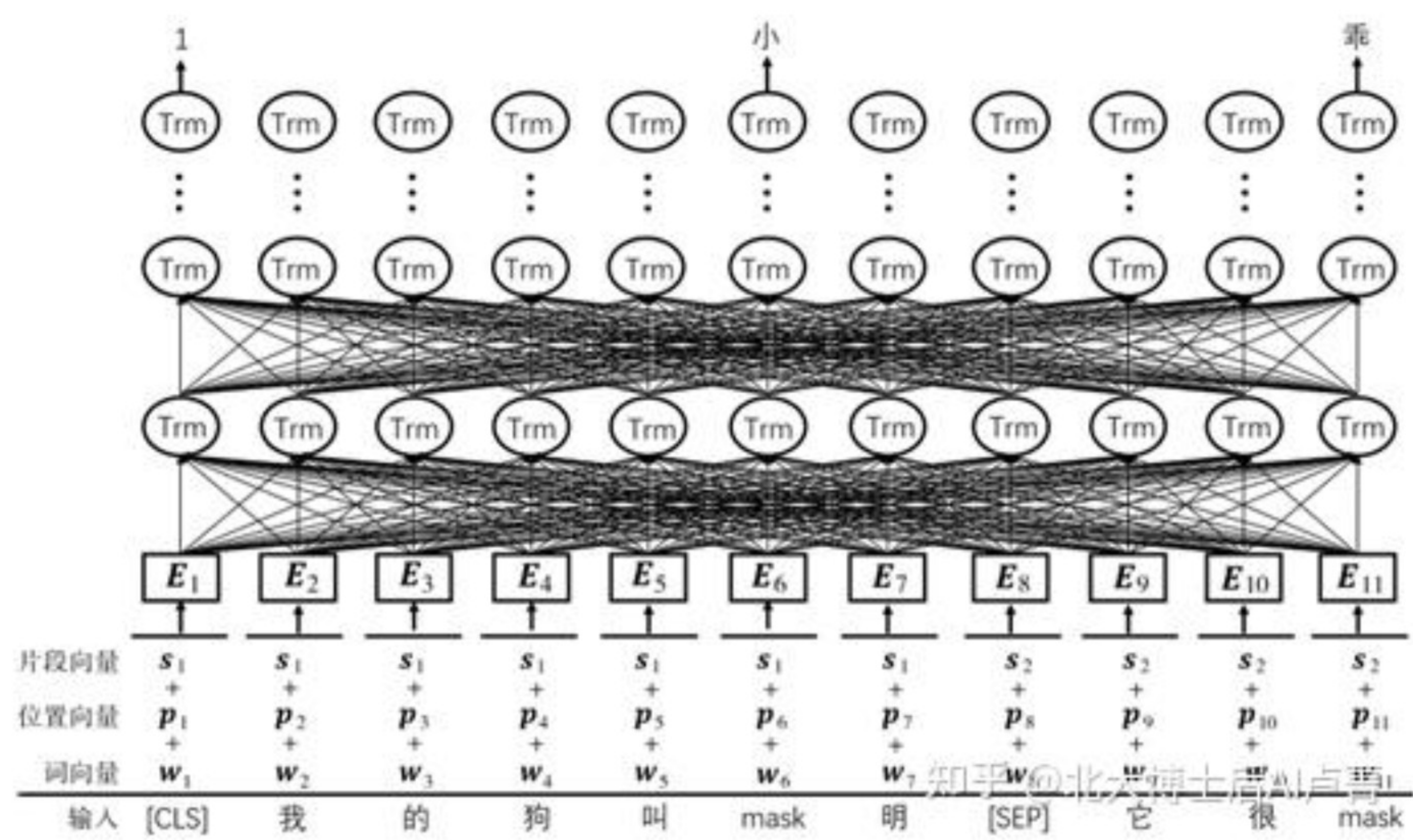


图14-15

使用训练好的BERT模型，我们可以方便地抽取句子的词向量和相应的句子向量，将其作为语义特征，提供给后续环节。

当业务语料专业性过强或任务特殊（例如金融等垂直领域），直接使用BERT模型效果不佳时，可采取以下两种改进方法。

通过语料进一步训练BERT模型。因为BERT模型的网络较深，所以，可冻结前层，仅训练靠近输出的后层。这种方法称为Fine-Tune。

将BERT模型的输出作为另一个多层网络的输入的特征，该多层网络的输出和业务的任务相关（使用垂直领域的语料，只训练这个多层网络）。此时，BERT模型以提取器的身份出现。在使用语料训练模型时，只需要改变该多层网络的参数，不需要改变BERT模型本身的参数。这种方法称为迁移学习（Transfer Learning）。

复杂度较高的模型通常会搭配迁移学习，主要原因在于迁移学习有以下优点。

“站在巨人的肩膀上”：前人花很大精力训练出来的模型，在大概率上会比我们从头开始搭建的模型强大。所以，我们没有必要“重复造轮子”，直接将BERT模型的输出作为特征即可。

训练成本低：如果采用导出特征向量的方法进行迁移学习，则只需要训练一个简单的多层网络，训练成本非常低（即使用CPU进行训练，也不会给机器带来压力）。

适用于小数据集：对于数据集本身很小（只有几千条数据）的情况，从头开始训练包含数千万个甚至上亿个参数的大型神经网络是不现实的（因为模型越复杂，需要的数据量就越大，所以无法避免过拟合）。此时，如果还想使用大型神经网络的超强特征提取能力，就只能依靠迁移学习了。

迁移学习是使用BERT模型时采用的策略。在图像处理领域，有ResNet、VGG等复杂的神经网络，这些网络也常作为特征提取器使用。

